Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 11
cs.pub.ro
cs.curs.pub.ro

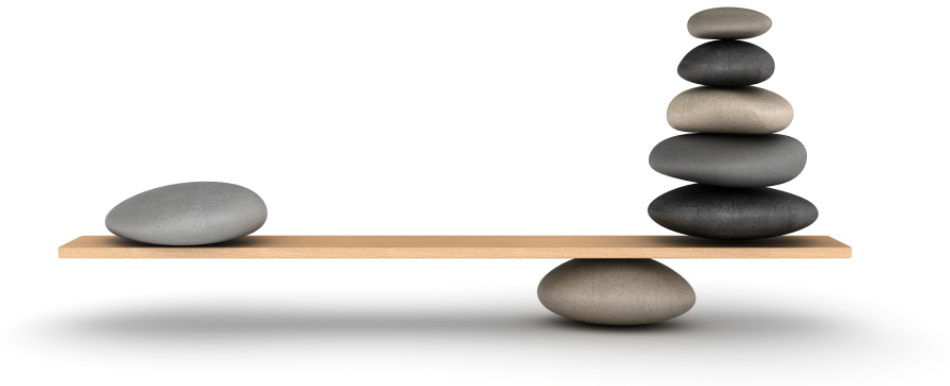
University Politehnica of Bucharest



Cuprins

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”





Nevoia de a Compara Sisteme de Calcul

Motivatie

- Lectura specificatiilor unor sisteme de calcul nu este suficienta pentru a ne oferi o imagine completa asupra performantelor acestora
- Asadar, a fost necesara crearea unor teste (benchmark-uri) care sa permita compararea performantelor intre sisteme/arhitecturi diverse
- Testele constau din seturi de programe care sunt menite sa scoata in evidenta performantele diverselor componente ale unui SC:
  - Floating Point CPU Performance
  - Caracteristici de interconectare (Bandwidth/Latenta)
  - Memorii externe (Hard-Drives)

Caracteristici & Probleme

- Sunt create pentru a “simula” comportamentul aplicatiilor reale (sintetice)
- Cele mai bune benchmark-uri sunt aplicatiile utilizator in sine…
- Trebuie sa fim intotdeauna circumspecti cand utilizam benchmark-uri!
- Cateva probleme:
  - Producatorii HW fac “tuning”
  - Performantele masinilor se degradeaza cand load-ul depaseste o valoare de prag
  - Securitatea, disponibilitatea, fiabilitatea, serviceabilitatea sau scalabilitatea NU sunt obiectul unui benchmark



Tipuri de Benchmark-uri

- Aplicatii reale: procesoare de text, aplicatii utilizator
- Kernel: contine doar parti esentiale dintr-o varietate de programe (i.e. Linpack), cu rezultate in MFlops
- Micro-benchmark-uri: dezvoltate de utilizatori pentru a testa componente ale sistemului
- Sintetice: se strang date statistice asupra operatiilor efectuate de aplicatii si se construiesc programe care sa respecte proportia aflata astfel (Whetstone/ Dhrystone)
- Pentru sisteme I/O
- Pentru sisteme paralele





Cum Masuram Performanta?

Latency vs. Throughput

- **Latency** (timpul de executie): timpul necesar terminarii unui task anume
- **Throughput** / bandwidth (latimea de banda): numarul de taskuri/transferuri intr-un timp fix
  - Exploateaza paralelismul pentru throughput si nu pentru latenta
  - Deseori latenta si throughputul pun presiuni contradictorii pe sisteme
  - Definitii diferite in functie de cod
    - Simulare Stiintifica - **Latency**
    - Web-Server - **Throughput**

Exemplu: Latency vs. Throughput

- Cum mutam oameni 10 km?
  - Masina: capacitate = 5; viteza = 60 km/h
  - Autobuz: capacitate = 60; viteza = 20 km/h
- Latency: **Masina = 10 min**; Autobuz = 30 min
- Throughput: Masina = 15 ppl/h; **Autobuz = 60 ppl/h**





Comparand Performanta - Speedup

- A este de X ori mai rapid ca B daca
  - X = Latency(B)/Latency(A)
    - Imparte la cel mai rapid
  - X = Throughput(A)/Throughput(B)
    - Imparte la cel mai incet
- A este cu X% mai rapid ca B daca
  - Latency(A) = Latency(B) / (1 + X/100)
  - Throughput(A) = Throughput(B) * (1 + X/100)

Exemplu: Latency vs. Throughput

- Latency
  - Masina este de **3** ori (si **200%**) mai "rapida" ca Autobuzul



- Throughput
  - Autobuzul este de **4** ori (si **300%**) mai "rapid" ca Masina



Speedup si % Crestere/Descrestere

- Programul A ruleaza 200 cicli - facand un task X
- Programul B ruleaza 350 cicli - facand acelasi task X
- % crestere si descrestere **nu sunt la fel**:
  - % crestere: ((350-200)/200)*100 = **75%**
  - % descrestere: ((350-200)/350)*100 = **42,3%**
- Speedup:
  - 350/200 = 1.75: A este de 1.75 ori mai rapid ca B
  - % crestere: (1.75 - 1)*100 = 75%
- Daca C e 1.5x mai rapid ca A, cati cicli ruleaza C?
  - 133 cicli, adica 50% mai rapid ca A

Calculul Performantei - Medii

- Media Aritmetica: (1/N) * $\sum_{P=1..N}$ Latenta(P)
  - Pentru marimi ce sunt proportionale cu timpul
- Media Armonica: N / $\sum_{P=1..N}$ 1/Throughput(P)
  - Pentru marimi ce sunt invers proportionale cu timpul
- Media Geometrica: $\sqrt[N]{\prod_{P=1..N}}$ Speedup(P)
  - Pentru marimi adimensionale



Exemplu de Medie Armonica

- Conduceti 2 km
  - 30 km/h pentru primul km
  - 90 km/h pentru al doilea km
- Care este viteza medie?
  - Atentie: raspunsul **nu este 60 km/h**
  - 0.033 h / km pentru km unu
  - 0.011 h / km pentru km doi
  - 0.022 h / km in medie = **45 km/h**



Performanta Procesoarelor vs. Workloads

- Q: Ce inseamna performanta unui procesor?
- A: Nimic fara un workload asociat.
- Benchmarks
  - Reprezinta un set standard de workload-uri
  - Utilizate pentru a compara performanta mai multor sisteme
  - Reprezentative programelor reale sau chiar programe reale
- Micro-Benchmarks
  - Set non-standard de non-workload-uri
  - Programe mici utilizate pentru izolarea unor chestiuni de performanta
  - Nereprezentative pentru manifestari complexe: turnuri Hanoi, 8-queens, etc.



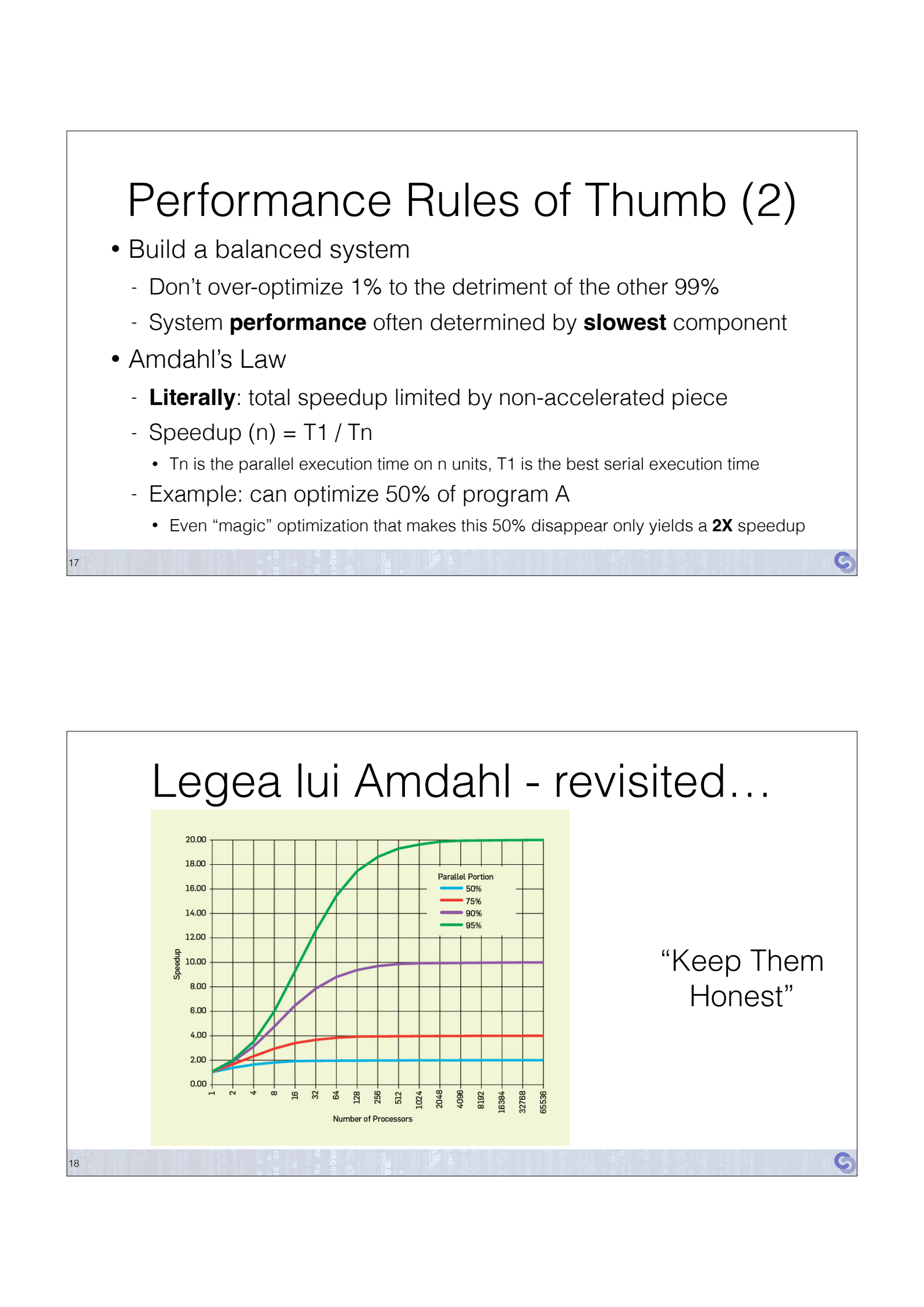
Performance Rules of Thumb

- Design for actual performance, not peak performance
  - Peak performance: “Performance you are guaranteed not to exceed”
  - Greater than “actual” or “average” or “sustained” performance
  - For actual performance X, machine capability must be > X
- Easier to “buy” bandwidth than latency
  - Which approach is easier, to transport more cargo via train?
    - Build another track?
    - Make a train that goes twice as fast?
  - Use bandwidth to reduce latency



Performance Rules of Thumb (2)

- Build a balanced system
  - Don't over-optimize 1% to the detriment of the other 99%
  - System **performance** often determined by **slowest** component
- Amdahl's Law
  - **Literally**: total speedup limited by non-accelerated piece
  - Speedup (n) = T1 / Tn
    - Tn is the parallel execution time on n units, T1 is the best serial execution time
  - Example: can optimize 50% of program A
    - Even "magic" optimization that makes this 50% disappear only yields a **2X** speedup



Legea lui Amdahl - revisited…

"Keep Them Honest"



Benchmark-uri pentru Masini “Seriale”

SPEC

- SPEC (Standard Performance Evaluation Corporation) realizeaza benchmark-uri pentru sisteme de calcul si le publica pe web
- Incearca sa simuleze situatii reale:
  - SPECweb testeaza performantele unui server de web pentru cereri HTTP paralele
  - SPEC CPU testeaza CPU-ul ruland 29 de aplicații INT si FP
- SPEC sunt scrise in C sau Fortran
  - Compilatoarele pot avea optimizari pentru acest tip de aplicatie
- Se distribuie cu o licenta
- Exista un conflict cu GPL (foloseste cod GPL)… http://www.spec.org/

Performanta SPEC

- Un sistem cu SPEC 150 este de 1.5 ori mai rapid decat un sistem cu SPEC 100
- Numarul SPEC generic se calculeaza din performanta celor 29 de aplicatii:
  - Medie geometrica
  - Medie aritmetica
  - Medie aritmetica ponderata in functie de prioritatile utilizatorului



GeekBench

- Benchmark-uri multicore multi-platforma
  - Laptop, workstation, iPhone, Android, etc.
- Performanta: Int, FP, Memorie, Bandwidth, etc.
- Salvarea rezultatelor online
  - Usor de comparat cu alte sisteme/procesoare
- Problema: workload-uri simple ce nu “streseaza” complet sistemele



Exemple GeekBench

- Laptop:
  - Intel “Ivy bridge” at 3.4 GHz (4 cores) 11,456
- Workstation
  - Intel Core i7-3520M at 2.9 GHz (2 cores) 7,807
- Platforme mobile:
  - iPhone 4S - Apple A5 at 0.8 GHz (2 cores) 642
  - iPhone 5 - Apple A6 at 1 GHz (2 cores) 1,589
  - Samsung G S3 - Snapdragon S3 – 1.5 GHz (2 cores) 1,429



TPC & BAPCo & Sandra

- TPC (Transaction Processing Performance Council) a definit benchmark-uri pentru procesarea tranzactiilor si a bazelor de date:
  - http://www.tpc.org/
- Consortiul BAPCo (Business Application Performance Corporation) este format din Intel, AMD, Dell, HP, NVidia, Apple & Microsoft, etc:
  - WEBmark, SYSmark, si MobileMark
  - http://www.bapco.com/
- SiSoftware Sandra (System ANalyser, Diagnostic and Reporting Assistant) este un program de testare si diagnoza:
  - http://www.sisoftware.net/





Benchmark-uri pentru Masini “Paralele”

The LINPACK Benchmark

- Linpack folosit pentru sisteme paralele pentru ca
  - Este extrem de raspandit si utilizat
  - Date si teste pentru acest program sunt disponibile pentru **toate** sitemele relevante din lume
- Benchmark-ul masoara cat de repede rezolva computer-ul un sistem dens de ecuatii liniare $A \cdot x = b$ de dimensiune $n \cdot n$
- Solutia se bazeaza pe metoda eliminarii Gaussiene, utilizand pivotarea partiala
- Conduce la $2/3 \cdot n^3 + n^2$ operatii in virgula mobila $O(n^3)$
- Se exclude utilizarea metodelor optimizate de inmultire a matricelor de genul metodei lui Strassen $O(n^{2.807})$ sau Coppersmith–Winograd $O(n^{2.376})$
- In acest mod se vor genera milioane de operatii in virgula mobila pe secunda (Mflop/s)
- Rezolvarea sistemelor de ecuatii liniare este o problema intalnita **frecvent** in aproape orice aplicatie stiintifica sau inginereasca

The LINPACK Benchmark (+)

- Avantaje:
  - Ofera ca rezultat un **singur** numar
    - Rmax = performanta maxima (**obtinuta**)
    - Rpeak = performanta maxima teoretica de varf (**estimata**)
  - Usor de definit si usor de clasificat pe baza lui
  - Permite modificarea dimensiunii problemei (n x n)
  - Ocupa sistemul testat cu un job de lunga durata
  - Disponibil la: http://www.netlib.org/benchmark/hpl/



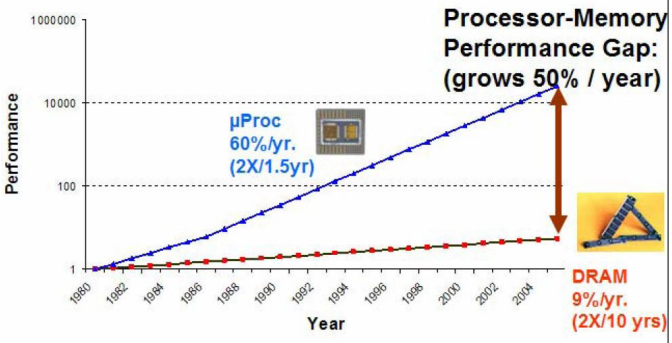
The LINPACK Benchmark (-)

- Dezavantaje
  - Scoate in evidenta **doar** viteza “peak” si numarul de procesoare
  - **Nu** evidentiaza (explicit) rata de transfer locala (bandwidth)
  - **Nu** evidentiaza (explicit) reteaua de comutare
  - **Nu** testeaza (explicit) mecanisme de tipul gather/scatter
  - Ignora legea lui Amdahl – permite doar scalare “slaba”
- Un singur numar nu poate caracteriza performanta totala a unui sistem de calcul!



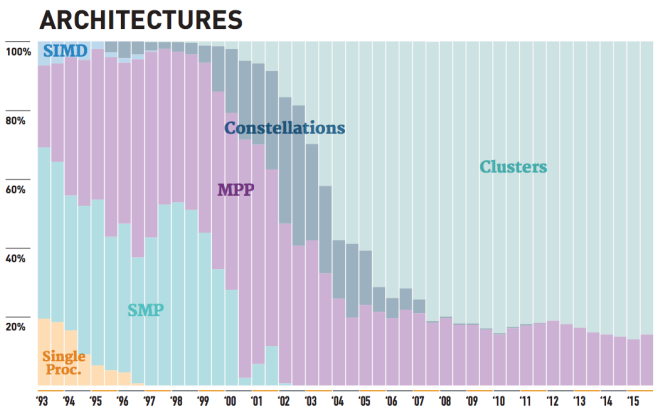
Relevanta LINPACK

- Daca luam in considerare anii 80'-90', poate LINPACK era cel mai potrivit benchmark
- Zidul memoriei era mai mult o treapta
- Memoria era mai "apropiata" de CPU:
  - n ciclii / executie instructiune
  - n ciclii / aducerea unui cuvant din memorie
- LINPACK arata de asemenea gradul de optimizare al compilatoarelor



Schimbari ale Sistemelor de Calcul

- In ultimii 40 de ani s-au schimbat foarte multe in hardware-ul sistemelor de calcul:
  - Arhitecturi vectoriale, superscalare, distribuite, cu memorie partajata, sisteme multicore, etc
- Cu toate ca LINPACK a fost modificat, nu toate modificarile reflecta progresele hardware
- Ierarhia memoriilor este mult mai complexa acum



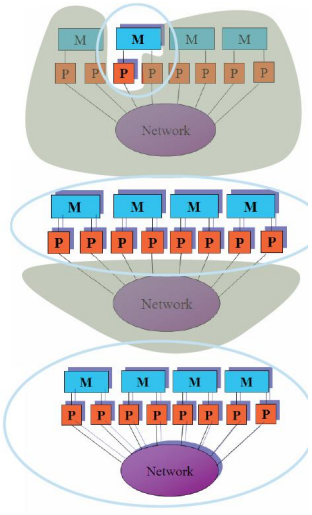
HPC Challenge Benchmark

- Un Benchmark ce evidentiaza
  - Procesoarele
  - Memoria
  - Reteaua de interconectare
- Avantajul major al HPCC
  - Arhitecturile pot fi descrise cu mult mai multe metrici decat FLOPS-urile LINPACK
- Scopuri ale HPCC
  - Complementar cu TOP 500
  - Exprimarea performatei aplicatiilor ca functie a caracteristilor accesului la memorie (localitate spatiala / temporala)
  - Permite optimizari
  - Inregistreaza efortul necesar pentru tuning-ul aplicatiilor
  - Rezultate arhivate si verificabile

HPCS = High Productivity Computing Systems
DARPA = Defense Advanced Research Projects Agency

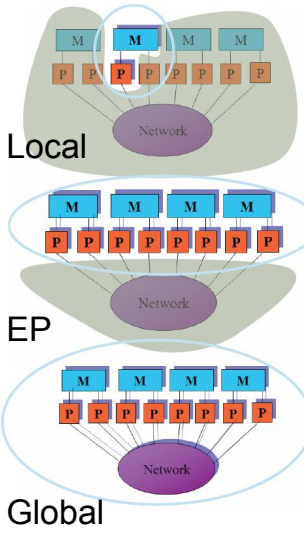
Testarea unui singur Procesor si a intregului Sistem de Calcul

- **Local**: un singur procesor participa la calculele dorite
- **Embarrasingly Parallel**: fiecare procesor din sistem participa la calcule, insa ele nu comunica explicit intre ele
- **Global**: toate procesoarele din sistem participa la calcule si comunica explicit intre ele

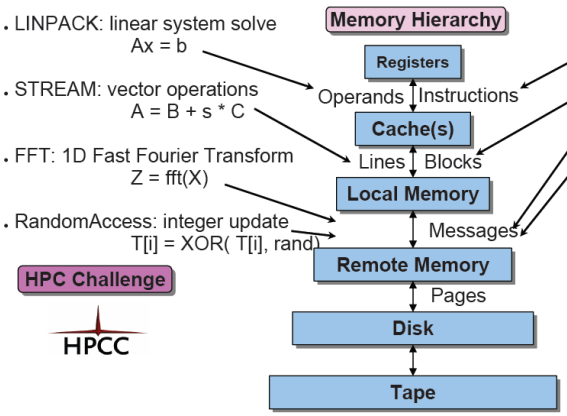


Componente HPCC

- HPCC e format din 7 benchmark-uri relevante:
  1. **HPL** (LINPACK) – MPI Global (Ax=b)
  2. **STREAM** – Local / ***STREAM** – EP (A ← B + s*C)
  3. **PTRANS** – MPI Global (A ← A + B^T)
  4. **RandomAccess** – Local, MPI Global / ***RandomAccess** – EP (T[i] ← XOR(T[i], rand))
  5. **Bandwidth** and **Latency** – MPI
  6. **FFT** – Local, Global & EP (Z ← fft(X))
  7. **Matrix Multiply** – Local & EP
- HPCC == framework pentru adaugarea de benchmark-uri



Scopul HPCC



Max	Relative
2 Pflop/s	8x
6.5 Pbyte/s	40x
0.5 Pflop/s	200x
64000 GUPS	2000x

DARPA HPCS

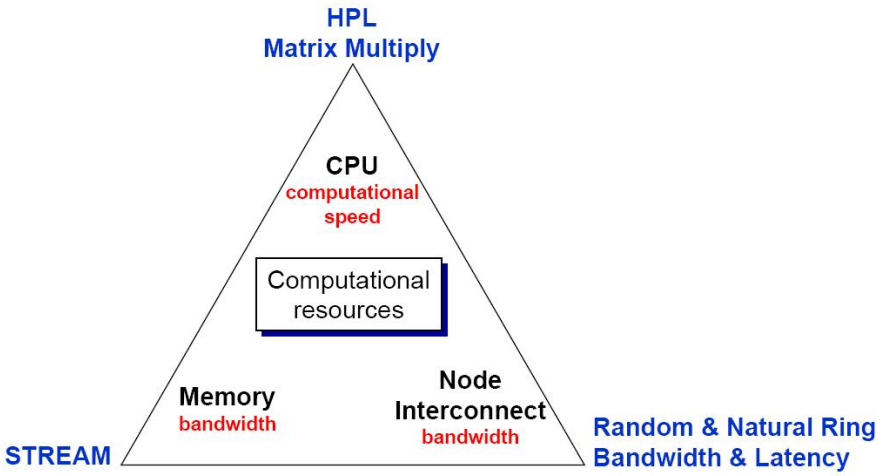
Performance Targets

- HPCS incearca prin HPCC sa:
  - Uniformizeze diferentele existente in ierarhia de memorie
  - Imbunatateasca performantele aplicatiilor reale
  - Usureze programarea sistemelor (paralele) de calcul

Functionarea HPCC

- Benchmark-ul functioneaza ca un program de sine statator
- Similar ca fisier de intrare/specificare cu HPL
- HPCC este format dintr-un “Base Run”, urmat de eventuale “Optimization Runs”:
  - Base Run este rularea Benchmark-ului “out of the box”
  - Utilizatorul trebuie sa instaleze pe masinile de test MPI si BLAS
  - In Optimized Runs se pot inlocui anumite subrutine, pentru a creste performantele unei anume componente
- Rezultatele se arhiveaza la: http://icl.cs.utk.edu/hpcc/
- Se pot genera apoi tabele html sau Excel cu rezultatele obtinute:
  - In mod intentionat nu se va oferi un singur criteriu de evaluare
  - Astfel NU se vor obtine clasificari generale!
- Scopul declarat al HPCC: sa NU necesite mai mult de dublul unei rulari HPL

Resurse vs HPCC



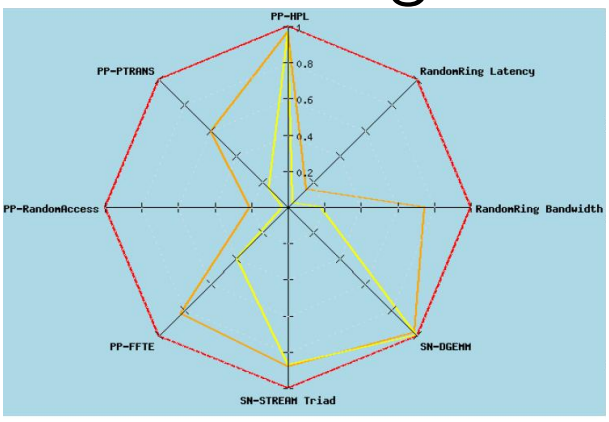
- Din ce sunt formate componentele HPCC?

Condensed Results - Base Runs Only - 114 Systems - Generated on Tue Nov 14 17:13:55 2006

System Information System - Processor - Speed - Count - Threads - Processes				G-HPL	G-PTRANS	G-Random Access	G-FFTE	EP-STREAM Sys	EP-STREAM Triad	EP-DGEMM	RandomRing Bandwidth	RandomRing Latency
MA/PT/PS/PC/TH/PR/CM/CS/IC/IA/SD				TFlop/s	GB/s	Gup/s	GFlop/s	GB/s	GB/s	GFlop/s	GB/s	usec
Cray Inc. Red Storm/XT3 AMD Opteron	2.4GHz	12960	125920	91.0350000	2336.9700	1.7401500	1334.0700	54840.499	2.1138	4.39939	0.05911	16.29
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	80.6830000	339.2840	0.0657312	2178.1100	53555.888	0.8172	1.85619	0.01084	8.84
IBM p5-575 Power5	1.9GHz	10240	110240	57.8670000	553.0090	0.1693440	842.5000	55184.179	5.3891	7.08562	0.11015	118.59
IBM p5-575 Power5	1.9GHz	8192	18192	45.7019000	2625.1700	0.3239760	908.6920	44455.936	5.4268	7.06423	0.08871	11.05
Cray Inc. XT3 Dual-Core AMD Opteron	2.6GHz	10404	110404	43.4033000	778.3850	0.8235630	1107.2100	25774.557	2.4774	4.78995	0.06937	14.32
IBM Blue Gene/L PowerPC 440	0.7GHz	65536	165536	37.3540000	4665.9100	0.1648600	1762.8200	62889.787	0.9596	2.47017	0.01039	8.62
IBM p5-575 Power5	1.9GHz	8192	28192	33.3175000	575.8230	0.2066390	965.6680	43802.460	5.3470	6.08616	0.07698	51.99
Cray Inc. XT3 AMD Opteron	2GHz	10350	110350	32.9665000	1813.0600	1.0176500	1118.2900	43581.780	4.2108	3.66719	0.16186	10.32
IBM Blue Gene/L PowerPC 440	0.7GHz	32768	132768	31.2581000	87.7818	0.2780090	1112.8100	29913.678	0.9129	2.17447	0.01197	9.51
Cray Inc. XT3 AMD Opteron	2.4GHz	5200	15200	20.5270000	874.8990	0.2685830	644.7300	26020.800	5.0040	4.39535	0.14682	25.80
Cray Inc. XT3 AMD Opteron	2.4GHz	5208	15208	20.4086000	944.2270	0.6724120	761.7290	24268.447	4.6598	4.41173	0.20636	9.20
Cray Inc. XT3 AMD Opteron	2.6GHz	4096	14096	16.9752000	302.9790	0.5330720	905.5690	20656.456	5.0431	4.78166	0.16896	9.44
Cray Inc. XT3 AMD Opteron	2.6GHz	4128	14128	16.6421000	674.7860	0.6767580	821.6770	19295.676	4.6743	4.75946	0.22245	8.23
Cray Inc. XT3 AMD Opteron	2.4GHz	3744	13744	14.7040000	608.5060	0.2202960	417.1720	18146.382	4.8468	4.41330	0.16164	25.32
Cray Inc. X1 Cray E	1.13GHz	1008	11008	12.0262000	108.0190	0.0861199	82.3884	15522.091	15.3989	14.50000	0.15667	16.30
HP XC Intel Itanium 2	1.6GHz	2048	12048	10.5616000	306.4190	0.2633320	339.8360	4000.031	1.9531	6.26484	0.15920	6.04

System Information System - Processor - Speed - Count - Threads - Processes				G-HPL	G-PTRANS	G-Random Access	G-FFTE	EP-STREAM Sys	EP-STREAM Triad	EP-DGEMM	RandomRing Bandwidth	RandomRing Latency
MA/PT/PS/PC/TH/PR/CM/CS/IC/IA/SD				TFlop/s	GB/s	Gup/s	GFlop/s	GB/s	GB/s	GFlop/s	GB/s	usec
SGI Columbia 2048 Intel Itanium 2	1.6GHz	2024	12024	9.3196500	18.1901	0.0491621	45.7760	3998.493	1.9755	6.23637	0.12271	6.98
NEC SX-8	2GHz	576	1 576	8.0085800	312.7070	0.0193617	160.9480	23555.750	40.8954	15.22320	0.82924	22.27
IBM p655 Power4+	1.5GHz	2048	12048	6.2094300	103.8040	0.1417230	136.5440	3631.636	1.7733	3.99073	0.06960	13.36
SGI Altix 3700 Bx2 Intel Itanium 2	1.6GHz	1008	11008	5.1383200	105.6660	0.0325982	15.6619	1907.509	1.8924	5.88404	0.20288	6.82
HP XC Intel Itanium 2	1.6GHz	1024	11024	4.8532700	154.1290	0.1615660	184.1270	2002.422	1.9555	6.24250	0.15978	5.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7823400	217.9230	0.1370020	266.6600	5274.698	4.7952	4.81050	0.28638	25.94
Cray Inc. XT3 AMD Opteron	2.6GHz	1100	11100	4.7276600	253.3460	0.3035680	328.2860	5161.134	4.6919	4.77440	0.39964	7.29
IBM P5 P575+ Power5+	1.9GHz	512	1 512	3.4100800	219.5560	0.0848539	52.2538	2997.745	5.8550	7.62342	0.05243	36.14
Cray Inc. X1E Cray X1 MSP	1.13GHz	252	1 252	3.1940900	85.2040	0.0148684	15.5352	2439.985	9.6825	14.18470	0.36024	14.93
IBM p655 Power4+	1.5GHz	1024	11024	3.1102600	51.4248	0.0983333	82.1033	1733.284	1.6927	3.72701	0.09024	10.13

Diagrame Kiviat

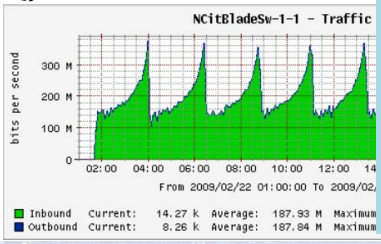


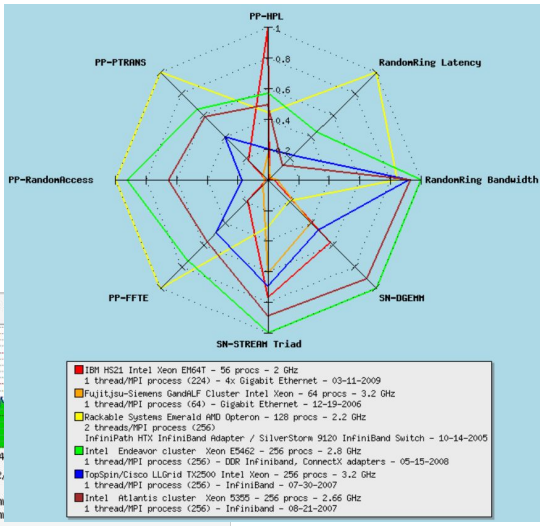
- HPL, STREAM, DGEMM: tehnologia de interconectarea nu conteaza; HPL scaleaza bine iar STREAM si DGEMM nu comunica (sunt locale)
- RandomRing Bandwidth: Cray-ul are cel mai mare bandwidth
- RandomRing Latency: Cray-ul are si cea mai mica latenta
- Random Access: Tehnologia de interconectare conteaza mult – senzitivitate la latenta
- PTRANS & FFTE: Tehnologia de interconectare conteaza mult – senzitivitate la bandwidth

HPL & HPCC @cs.pub.ro

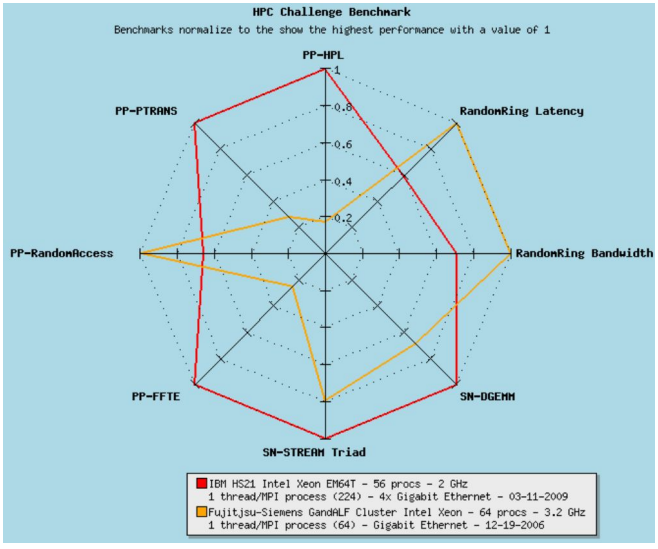
- In 2009 we broke the 1TFlops barrier with 28 of our 32 IBM HS21 blades (2xQuad-Core)
- HPL performance:
  - **1003 GFlops Max**
  - 1792 GFlops Peak
- System dimension:
  - 230.000 (Ax=b)

55.88%





Interconectare in Clustere cs.pub.ro



Dezvoltarea Viitoare a HPCC

- Proiectul doreste crearea unui cadru pentru dezvoltearea de benchmark-uri
- Se doreste dezvoltarea unor “semnaturi” ale fiecarei masini
- In viitor, colectia de benchmark-uri trebuie marita
  - De exemplu pentru operatii cu matrici rare
- Portarea HPCC pe cat mai multe sisteme
- Implementarea diversa a HPCC
  - In noi limbaje: UPC, Co-Array, Chapel, Charm++,…
  - In noi medii si paradigme: Spark, 0MQ, Flink,…



Alte Benchmark-uri Paralele

- SPLASH2 - Stanford Parallel Applications for Shared Memory
  - http://www.capsl.udel.edu/splash/
- NAS Parallel Benchmark
  - https://www.nas.nasa.gov/publications/npb.html
- SPECopenMP
  - Parallelized (OpenMP) versions of SPECfp 2000
- SPECjbb
  - Java multithreaded database-like workload

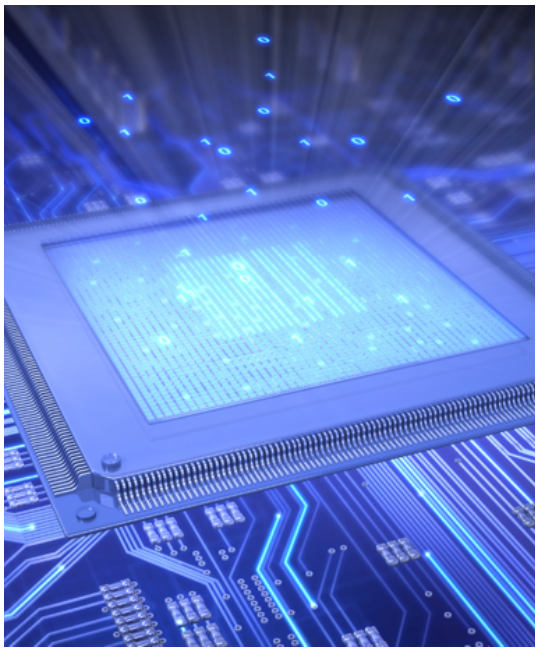


Ce am invatat in aces curs?

- Nevoia de a compara sisteme de calcul
- Cum masuram Performanta?
- Benchmark-uri pentru masini “seriale”
- Benchmark-uri pentru masini “paralele”

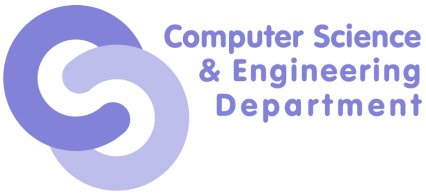
Slides & Support Material Acknowledgements

CIS 371 - University of Pennsylvania 2013
CS/ECE 3810 - University of Colorado 2017
University of Tennessee 2017
http://www.netlib.org 2017



Thank you for your attention.





emil.slusanschi@cs.pub.ro
cs.pub.ro